



数算期末整理

Created @December 22, 2025 8:27 PM

第七章 图

关于定义

稀疏：非0元数量占比<5%

存储：

存储方式	空间复杂度	适用场景	核心原因
邻接矩阵	$O(n^2)$	稠密图	稠密图边多，矩阵的 $n \times n$ 空间能被充分利用（不会大量存 “0” 浪费空间），且查询任意两点是否有边的时间是 $O(1)$ ，效率高。
邻接表	$O(n+e)$	稀疏图	稀疏图边少，邻接表只存实际存在的边，空间开销小（避免了邻接矩阵 “存

存储方式	空间复杂度	适用场景	核心原因
			大量 0" 的浪费), 且遍历某顶点的邻接边更高效。

周游：深度，宽度优先；

周游的逻辑是已经访问过的节点不再展开→这样有生成树

拓扑排序：对于给定图，找出若干个或所有拓扑序列。任何有向无环图，都可以拓扑排序。

几个算法：

最短路：

单源dijkstra，逐个点加入

全部floyd(动归，3层

构建最小生成树：

Prim:类似dijkstra

Kruskal:搜最短

第八章 排序

算法汇总：

不稳定：快，选，shell，堆

1. 重点排序算法：直接插入法、★Shell排序、★快速排序、★基数排序、归并排序

2. 算法分析

1) 基于比较次数和移位次数分析最好、最坏的时间、空间：直接插入法、二分法插入排序、起泡排序、直接选择、快速排序、基数排序、归并排序

2) 记住各种排序方法的平均时间

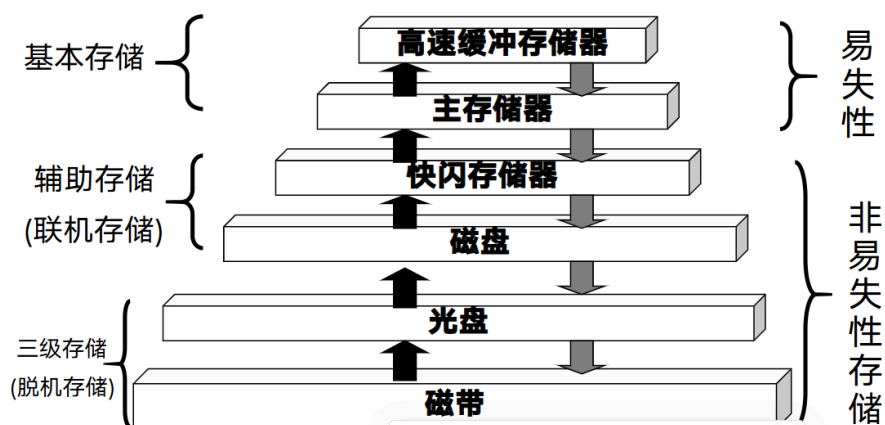
各种排序方法的局部修改和混合应用

第九章 文件与外存储

1.主存与外存

主存简称内存

区分一下多级存储：



~~磁盘：~~

1/3

~~文件：~~

~~缓冲区：~~

~~以上都在cue+CS-Cache..~~

不咋考)

外排序部分

1. 置换选择排序，基于最小堆

顺序，（假如最小堆就是从小往大输入）的话，那么就只需一个顺串；

如果反着来，从大往小：需要 n/m （ m 为缓冲区长度）

2. 多路归并，败者树

多路归并的核心是每次选一些顺串来合成长顺串，那么怎么选呢？

很符合直觉就是用Huffman K叉树；（这也就是“最佳归并树”

- 注意：需要补充0节点，使得其最后满足 $n-1$ 被 $k-1$ 整除
- 如果问总读写次数，记得 $\times 2$

败者树：

败者树：多路归并的“选最小元素加速器”

1. 定义

败者树是一棵 **完全二叉树**，用于在 k 个“候选元素”中快速找到最小值，核心特征：

- 叶子节点：存储 k 个归并段的“当前首元素”（候选元素）。
- 非叶子节点：存储“比赛败者”（即两个子节点中值较大的那个元素对应的归并段编号）。
- 根节点：最终败者（对应的胜者就是 k 个元素中的最小值）。
- 辅助节点（可选）：记录“全局胜者”（最小值），避免每次从根节点反向查找。

• 2. 用途

将多路归并中“选最小元素”的时间复杂度从 $O(k)$ 优化到 $O(\log k)$ ，大幅提升归并效率（ k 越大，优化效果越明显）。

工作原理

假设 $k=3$ 路归并，归并段 $R1=[2,5,8]$ 、 $R2=[1,4,7]$ 、 $R3=[3,6,9]$ ，败者树构造与查找过程：

- 初始阶段：叶子节点存 3 个归并段的首元素（2、1、3），非叶子节点进行“两两比赛”：
 - 左子树：2 vs 1 $\rightarrow$ 败者是 2（归并段 1），胜者是 1（归并段 2）。
 - 右子树：3 vs 胜者 1 $\rightarrow$ 败者是 3（归并段 3），根节点存败者 3。
 - 全局胜者：1（归并段 2 的元素），输出到结果。
- 替换阶段：R2 的下一个首元素是 4，替换叶子节点中原来的 1，然后从叶子向上“重赛”：
 - 叶子节点（R2）的 4 vs 左子树原败者 2 $\rightarrow$ 败者是 4（归并段 2），胜者是 2（归并段 1）。

- 胜者 2 vs 根节点原败者 3 → 败者是 3（归并段 3），根节点仍存 3。
 - 全局胜者：2（归并段 1 的元素），输出到结果。
- 重复替换 + 重赛，直到所有元素归并完成。
- 每次替换元素后，仅需从叶子节点向上“重赛”到根节点，共 $\log_2 k$ 次比较（线性查找是 k 次）。
- 与堆排序的区别：堆是“胜者树”（父节点是胜者），但败者树在多路归并中更高效——因为败者树的非叶子节点存储的是“归并段编号”，而非元素值，替换元素时无需修改非叶子节点的元素值，仅需比较编号对应的元素。

时间复杂度

- 原始方法： $O(nk)$
- 败者树方法： $O(k + n \log k)$

为什么不用堆？

堆的每层调整都要对三个值（父节点、两个子节点）进行至少两次比较，使用胜者败者树少一次比较，败者树相对胜者树更少一次寻址

第十章 检索

1. 平均检索长度
2. 二分搜索，判定树：普通二叉搜索树的结构是**不唯一的**（取决于元素的插入顺序），而且可以是不平衡的；但判定树是**基于有序数组的二分查找构建的，结构唯一且必然平衡**。
2. 散列概念；

散列表

- **负载因子** $\alpha = \text{填入表中的结点 } n / \text{散列表空间大小 } m$
- **冲突**：不同关键码计算出了相同的散列地址
 - 现实中不冲突的散列函数极少存在
 - **同义词**：产生冲突的两个关键码

3. 散列删除

删除

两点需要考虑:

- 删除后, 该地址的记录是否会影响检索
- 释放存储位置能为将来所用
 - 只有开散列方法可以真正删除, 空间重用

- 闭散列方法都只能作标记, 不能真正删除, 空间未再次分配之前不可用
- **墓碑**：用一个特殊的值表示删除
 - 不能将空单元与删除单元混淆：检索时遇到已删除标记还需要继续检索下去
- 删除操作与检索类似, 但是删除后地址标记为删除
- 带墓碑的插入：可以插, 但是检索要继续, 防止插入两个相同的关键码

第十一章 索引

1.B树

检索，本质是高度的性质

1. 在一次检索中涉及的节点最多的情况即从树的根部检索到深度最大的叶节点处。
该情况下，涉及的节点个数等于树的高度。一颗包含n个关键码的m阶B-树，其最大高度为 $h = \log_{\lceil m/2 \rceil} (\frac{n+1}{2}) + 1$ 。(推导过程如下)

$$\begin{aligned} n &\geq 1 * 1 + 2 * (\lceil m/2 \rceil^0 + \lceil m/2 \rceil^1 + \lceil m/2 \rceil^2 + \dots + \lceil m/2 \rceil^{h-2}) * (\lceil m/2 \rceil - 1) \\ &\geq 1 + 2 * (\frac{1 - \lceil m/2 \rceil^{h-1}}{\lceil m/2 \rceil - 1}) * (\lceil m/2 \rceil - 1) \\ &\geq 1 + 2 * (\lceil m/2 \rceil^{h-1} - 1) \end{aligned}$$

$$\begin{aligned} \lceil m/2 \rceil^{h-1} &\leq \frac{n+1}{2} \\ h &\leq \log_{\lceil m/2 \rceil} (\frac{n+1}{2}) + 1 \end{aligned}$$

2. 在一次检索中最少只涉及到一个结点。

B树插入就是溢出就中间元素往上移，

删除不够了就找邻居借，差的多了就和邻居合并

B+树多一次复写：

- 是B 树的一种变形，在叶结点上存储信息
 - 所有的关键码均出现在叶结点上
 - 各层结点中的关键码均是下一层相应结点中最大关键码（或最小关键码）的复写

B+树的插入：与B树类似, 左 $\lceil m/2 \rceil$, 右 $\lfloor m/2 \rfloor$, 注意上层关键码的维护

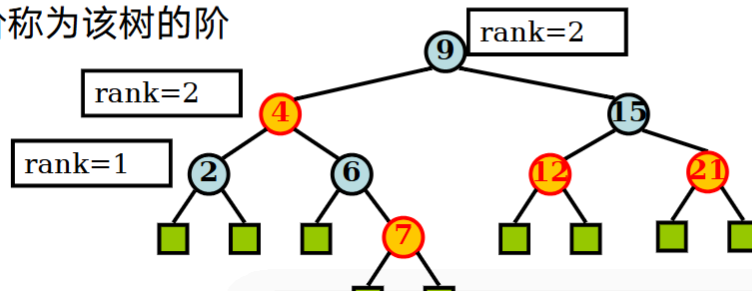
两棵树分裂时都是左节点 $\lceil m/2 \rceil$ 个子节点

B+树的删除：与B树类似, 但被删除关键码的上层副本可以选择保留, 作为"分界关键码"存在

红黑树

红黑树的阶

- 结点X的阶 (rank, 也称“黑色高度”)
 - 从该结点到外部结点的黑色结点数量
 - 不包括 X 结点本身, 包括叶结点
- 外部结点的阶是零
- 根的阶称为该树的阶



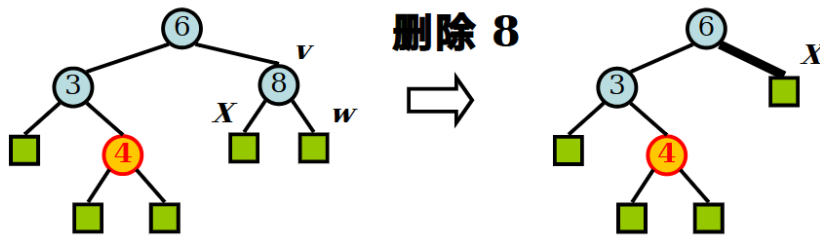
插入：

注意其实从定义出发, 基本策略不难想 (希望如此。

删除***：

11.6.4 删除算法

- 先调用 BST 的删除算法
 - 待删除的结点有一个以上的外部空指针，则直接删除
 - 否则在右子树中找到其后继结点进行值交换（着色不变）删除
- v 是被删除的内结点， w 是被删外结点， X 是 w 的兄弟
 - 如果 v 或者 X 是红色，则把 X 标记为黑色即可
 - 否则， X 需要标记为双黑，根据其兄弟结点 C 进行重构调整



第一步：先明确红黑树删除的核心术语（你的图里的概念）

- **内结点**：有子结点的实际结点（比如图里的 6、3、8、4）；
- **外结点 (w)**：红黑树里的“空结点 (NIL)”，默认是黑色（用来填充叶子位置，保证所有路径都到外结点结束）；
- **v**：被删除的**内结点**（你的图里是 8）；
- **X**：被删外结点 w 的**兄弟结点**（你的图里是 8 的另一个外结点）；
- **双黑**：删除黑色结点后，某条路径的“黑结点数减少”，此时把对应的外结点标记为“双黑”（表示需要额外补一个黑色）。

第二步：红黑树删除的总流程

红黑树删除 = “先按 BST 规则删结点” + “再修复红黑树性质”（删除可能破坏“黑高相等”“无红红冲突”等性质）。

阶段 1：按 BST 的规则删除结点

红黑树是二叉搜索树 (BST)，所以删除逻辑和 BST 完全一致，分 3 种情况：

1. 待删结点是“叶子内结点”（只有外结点，比如你的图里的 8）：直接删除该内结点，留下它的外结点（比如图里删除 8 后，原来的位置变成外结点 X）；
2. 待删结点只有 1 个实际子结点：用子结点代替待删结点，再删待删结点；
3. 待删结点有 2 个实际子结点：找它的后继结点（右子树的最小结点），交换待删结点和后继结点的值，再删除后继结点（后继结点一定是“叶子内结点”或“只有 1 个子结点”，回到情况 1/2）。

阶段 2：修复红黑树的性质（核心！解决“黑高不足”）

删除后最容易破坏的是**“黑高相等”性质**（如果删除的是黑色结点，某条路径的黑结点数会减少），需要根据情况修复。

你的图对应的是最简单的修复情况，我们结合规则讲：

情况 1：如果“被删内结点 v”或“w 的兄弟 X”是红色

- 场景：比如你的图里，被删的 v（8）是红色，或者 X 是红色；
- 操作：把 X 标记为黑色；
- 原理：原来 X 是红色，变黑后，X 所在路径的黑结点数增加 1，刚好弥补“删除黑色结点导致的黑高减少”，直接修复完成。

情况 2：如果“v 和 X 都是黑色”（更复杂，但考试不常考细节）

- 场景：v 和 X 都是黑色，删除后黑高不足；
- 操作：把 X 标记为双黑（表示需要补黑），再根据 X 的兄弟结点 C 的颜色，分“C 是红色”“C 是黑色且子结点颜色不同”等情况，通过调色 + 旋转补回黑高（比如把兄弟结点的黑色“借”过来）。

结合你的图，具体走一遍删除流程

你的图是“删除内结点 8”：

1. **BST 删除**：8 是叶子内结点（只有外结点），直接删除 8，留下它的外结点 X（绿色方块）；
2. **修复红黑树**：
 - 检查 v（8）或 X 的颜色：假设 8 是红色（或 X 是红色）；
 - 按规则：把 X 标记为黑色；

- 结果：X 所在路径的黑结点数补回，红黑树性质恢复（最终图如右边所示）。

红黑树删除的核心逻辑总结

- 删结点：先按 BST 删，只删“叶子内结点”或“后继结点”；
- 修性质：核心是**补黑高**——如果能通过“调色（把红结点变黑）”补，就直接调；调不了就标记双黑，再通过旋转 + 调色补。

第十二章 高级数据结构

广义表

基本概念

- 回顾线性表
 - 由 n ($n \geq 0$) 个数据元素组成的有限有序序列
 - 线性表的每个元素都具有相同的数据类型
- 如果一个线性表中还包括一个或者多个子表，那就称之为广义表（Generalized Lists，也称Multi-list）一般记作：

$$L = (x_0, x_1, \dots, x_i, \dots, x_{n-1})$$

head () 函数：取“第一个元素”（核心：不分原子 / 子表）

tail () 函数：取“剩余元素组成的新广义表”（核心：返回值必是表）

- **纯表** (pure list)
 - 从根节点到任何叶节点只有一条路径
 - 任何元素 (原子, 子表) 只能出现一次
- **可重入表(再入表)** (reentrant list)
 - 任何元素可以出现多次
 - 对应一个DAG
- **循环表(递归表)** (circular list)
 - 包含回路, 深度无穷大

图 $\supseteq$ 再入表 $\supseteq$ 纯表(树) $\supseteq$ 线性表

应用: Lisp语言中的S表达式, 函数调用关系, 内存引用关系

字符树

trie树

patricia (做路径压缩)

最佳二叉搜索树

度量标准是加权最短路径,

成功是层数加1, 失败就是层数, 然后加权平均

AVL

核心就是不能有左右子树差1层以上, 不然就转来调整

插入删除时间复杂度 $\log N$

伸展树

伸展树是一种**自调整的二叉搜索树**, 核心逻辑是“让刚访问过的节点靠近根”, 从而优化频繁访问节点的查找效率 (适合“访问频率不均”的场景, 比如热门数据查询)。它不追求“结构永久平衡” (像 AVL / 红黑树), 而是通过“访问后调整结构”实现“均摊高效”。

一、伸展树的核心目标

- 不是保持“左右子树高度差”，而是利用**局部性原理**：刚被访问的节点，大概率会被再次访问。
- 每次访问（查找、插入、删除）一个节点后，通过**旋转操作**（伸展），把这个节点移到**根附近**（最终移到根）。
- 优势：实现简单（不用维护复杂的平衡条件），对“热点节点”的查找效率极高。

二、核心操作：伸展（Splay）—— 三种旋转类型

“伸展”是把目标节点移到根的过程，依赖 3 种旋转（旋转时保持二叉搜索树的有序性）：

1. 单旋转（适用于“目标是根的直接子节点”）

当目标节点是根的**左孩子 / 右孩子**时，只需 1 次旋转（左旋 / 右旋），把目标转到根。

例子：根是 A（关键字 10），左孩子是 B（关键字 5），访问 B 后单旋转（右旋）：

- 旋转前：A（根）→ 左 B
- 旋转后：B（根）→ 右 A（旋转后仍满足二叉搜索树： $B < A$ ）

2. 一字形旋转（同方向旋转，适用于“左左 / 右右结构”）

当目标节点是“根的左孩子的左孩子”（左左），或“根的右孩子的右孩子”（右右）时，需 2 次**同方向**旋转。

例子（左左结构）：A（根，20）→ 左 B（10）→ 左 C（5），访问 C 后一字形旋转：

1. 第一次右旋 A：B 变成根，A 变成 B 的右孩子；
 2. 第二次右旋 B：C 变成根，B 变成 C 的右孩子；
- 最终：C（根）→ 右 B → 右 A（ $C < B < A$ ，保持有序）

3. 之字形旋转（异方向旋转，适用于“左右 / 右左结构”）

当目标节点是“根的左孩子的右孩子”（左右），或“根的右孩子的左孩子”（右左）时，需 2 次**异方向**旋转。

例子（左右结构）：A（根，20）→ 左 B（10）→ 右 C（15），访问 C 后之字形旋转：

1. 第一次左旋 B：C 变成 B 的父节点，B 变成 C 的左孩子；
2. 第二次右旋 A：C 变成根，A 变成 C 的右孩子；

- 最终：C（根）→ 左 B、右 A（ $B < C < A$ ，保持有序）

三、伸展的完整过程

无论目标节点在树的哪个位置，从目标节点开始，向上判断“当前节点与父 / 祖父的结构”（左左 / 右右→一字形，左右 / 右左→之字形，直接子节点→单旋转），依次执行旋转，直到目标节点移到**根的位置**。

四、简单应用场景

伸展树适合“访问有局部性”的场景：

- 数据库中的**热门数据索引**：频繁查询的记录会被移到根附近，查询更快；
- 缓存系统的**热点 Key 存储**：热点 Key 的查找开销极低；
- 文本编辑器的**最近操作历史**：最近编辑的内容对应的节点靠近根，再次操作更高效。

五、变种：半伸展树

半伸展树是伸展树的**简化版**，为了减少“频繁旋转的开销”：

- 普通伸展树会把目标节点移到根；
- 半伸展树只把目标节点**上移 1 层**（或只做 1 次旋转），不强制移到根；
- 特点：调整开销更小，但“热点节点靠近根”的效果弱于普通伸展树，是“调整开销”和“访问效率”的折中。

考试速记

- 伸展树核心：**访问后旋转，让目标节点靠近根**；
- 三种旋转：单旋（直接子节点）、一字形（左左 / 右右）、之字形（左右 / 右左）；
- 应用：**热点节点频繁访问的场景**；
- 半伸展树：简化版，只上移 1 层，开销更小。

★1. B树、B+树的插入 (注意保持性质，特别是等高；以及子结点和关键码个数的上下限制)

★2. B树/B+树的读盘和写盘次数分析

1. B树/B+树的效率分析

B树中关键码没有重复，父结点中的关键码是其子结点的分界；B+中最底层是关键码的一个全集，往根的方向一层层复写。

B树插入：插入 ----- 分裂

B+树插入：插入 ----- 分裂

★4. 红黑树的插入/删除方法

AI

一、B 树（多路平衡查找树）—— 外存随机查找核心

1. 先记 B 树的 3 个核心性质（插入 / 分裂的前提）

设 B 树的阶为 m ($m \geq 2$ ，考试常考 $m=3/4$ 阶)，满足：

- 关键码个数：每个节点的关键码数 k 满足 $\lceil m/2 \rceil - 1 \leq k \leq m - 1$ （下限：最小关键码数，上限：最大关键码数）；
- 子节点个数：每个节点的子节点数 c 满足 $\lceil m/2 \rceil \leq c \leq m$ （子节点数 = 关键码数 + 1）；
- 结构：所有叶子节点在**同一层**（等高），根节点至少有 2 个子节点（除非树只有 1 个节点）；
- 关键码有序：节点内关键码升序排列，父节点的关键码是子节点的“分界”（左子树关键码 < 父关键码 < 右子树关键码）。

考试速记：m 阶 B 树的“上下限”

- 最小关键码数： $\min_k = \lceil m/2 \rceil - 1$ （比如 $m=3 \rightarrow 1$ ， $m=4 \rightarrow 1$ ， $m=5 \rightarrow 2$ ）；
- 最大关键码数： $\max_k = m - 1$ （比如 $m=3 \rightarrow 2$ ， $m=4 \rightarrow 3$ ）；
- 触发分裂条件：节点关键码数 = $\max_k + 1$ （溢出）。

2. B 树的插入流程（核心：插入→溢出→分裂，保持等高）

步骤：

1. **查找插入位置**：从根节点出发，按关键码有序性，找到待插入的“叶子节点”（B 树插入只在叶子节点进行）；
2. **插入关键码**：在叶子节点中按升序插入新关键码，保持节点内有序；
3. **判断是否溢出**：若插入后节点关键码数 $\leq \max_k$ (2)，插入结束；若 $> \max_k$ （比如插入后 $k=3$ ），触发**分裂**；
4. **分裂操作（考试重点）**：
 - 取节点的**中间关键码**（第 $\lceil k/2 \rceil$ 个， $k=3$ 时取第 2 个）；

- 中间关键码**上移**到父节点（若父节点溢出，重复分裂，直到根节点）；
- 原节点分裂为左、右两个子节点：左子节点存中间关键码左边的所有关键码，右子节点存右边的所有关键码；
- 若分裂的是根节点（无父节点），则新建一个根节点，中间关键码作为根节点的唯一关键码，左、右子节点作为根的两个子树（树高 + 1，但仍保持等高）。

例子（m=3 阶 B 树插入关键码 5、3、7、2、4）：

- 初始插入 5（根节点，k=1，正常）；
- 插入 3（根节点，k=2，正常，节点：[3,5]）；
- 插入 7（根节点，k=3→溢出，分裂）：
 - 中间关键码 5 上移为新根，左子节点 [3]，右子节点 [7]（树高 = 2，等高）；
- 插入 2（找左子节点 [3]，插入后 k=2，正常，节点：[2,3]）；
- 插入 4（找左子节点 [2,3]，插入后 k=3→溢出，分裂）：
 - 中间关键码 3 上移到父节点 [5]，父节点变为 [3,5]（k=2，正常）；
 - 左子节点分裂为 [2]，右子节点分裂为 [4]；
- 最终 B 树：根 [3,5]，左子 [2]、[4]，右子 [7]（所有叶子在第 2 层）。

记忆口诀：B 树插入“三步曲”

“找叶子，插关键，溢了就分裂，中间上移，父节点接住，根分裂树高 + 1，始终等高”。

二、B + 树（B 树的优化版，外存范围查询核心）

1. B + 树与 B 树的核心区别

特性	B 树	B + 树
关键码存储	非叶子节点 + 叶子节点都存关键码	仅叶子节点存 全部关键码 （全集），非叶子节点存索引（关键码副本）
父节点与子节点关系	父关键码是子节点的分界（不重复）	父关键码是子节点的最大 / 最小关键码（重复存储）
叶子节点	孤立，无关联	按顺序连成 双向链表 （方便范围查询）
查找终点	非叶子 / 叶子节点（找到即返回）	必须到叶子节点（无论成功失败）

“叶子存全集，非叶是索引，叶子链表连，范围查询快”。

2. B + 树的插入流程（和 B 树类似，差异在分裂）

设 m 阶 B + 树（关键码数范围同 B 树： $\lceil m/2 \rceil - 1 \leq k \leq m - 1$ ），插入流程：

1. **查找插入位置**：必须找到**叶子节点**（唯一插入位置）；
2. **插入关键码**：在叶子节点中按升序插入，保持有序，同时更新叶子节点的链表指针；
3. **判断溢出**：若插入后 $k \leq \max_k$ ，结束；若 $k > \max_k$ ，触发分裂；
4. **分裂操作（核心差异）**：
 - 取中间关键码（第 $\lceil k/2 \rceil$ 个）；
 - 中间关键码**复制到父节点**（不是上移！因为叶子节点要保留该关键码，保证全集完整）；
 - 原叶子节点分裂为左、右两个叶子节点，左存中间左边关键码，右存中间右边关键码，且新叶子节点接入链表；
 - 若父节点溢出，重复分裂，直到根节点（根分裂时，中间关键码成为新根的索引，树高 + 1，保持等高）。

考试重点：B + 树分裂 vs B 树分裂

- B 树：中间关键码“上移”（子节点不再保留）；
- B + 树：中间关键码“复制”（子节点仍保留，父节点只存索引）。

三、B 树 / B + 树的读盘 / 写盘次数分析（外存 IO 核心考点）

外存中，B 树 / B + 树的**每个节点对应一个磁盘块**，访问一个节点 = 1 次 IO（读盘 / 写盘），IO 次数 = 树的高度（或分裂时的节点数）。

1. 读盘次数（查找操作）

- 查找成功：B 树最多读 **树高** 次（找到节点即返回）；B + 树必须读 **树高** 次（到叶子节点）；
- 查找失败：两者均读 **树高** 次（B 树到空指针，B 树到叶子节点的空位置）；
- 核心逻辑：树高 = IO 次数，B 树 / B + 树的树高是 $O(\log_m n)$ （ m 是阶数， n 是关键码总数），比如 $m=100$ 阶， $n=1e6$ 时，树高 $\approx \log_{100} 1e6 = 3$ ，仅需 3 次 IO（远优于二叉搜索树的 $O(n)$ IO）。

2. 写盘次数（插入操作）

- 正常插入（不溢出）：1 次写盘（仅修改叶子节点）；
- 溢出分裂：写盘次数 = 分裂的节点数 + 1（分裂的子节点 + 父节点）；
 - 例：m=3 阶 B 树，插入后叶子节点分裂，父节点也溢出分裂，根节点分裂→写盘次数 = 2（分裂的叶子）+1（父节点）+1（新根）=4 次；
- B + 树分裂写盘次数同 B 树，但需额外更新叶子链表指针（1 次额外写盘）。

记忆：IO 次数 = 树高，分裂多写盘，B + 树链表要多写一次。

四、B 树 / B + 树的效率分析（考试选择题高频）

场景	B 树	B + 树
随机查找（单个关键码）	效率略高（可能提前结束）	效率相当（必须到叶子，但索引层更紧凑）
范围查找（比如 10~50）	低效（需回溯父节点）	高效（叶子链表直接遍历，无需回溯）
外存适配性	较好	更好（叶子链表 + 索引层紧凑，IO 更少）
插入删除效率	中等（分裂时中间关键码上移）	中等（分裂时复制关键码，写盘略多，但范围查询补偿）

考试结论：

- 数据库索引（外存）优先用**B + 树**（支持范围查询，IO 效率高）；
- 少量随机查找场景用**B 树**（写盘略少）。

五、红黑树（二叉平衡查找树，内存查找核心）

1. 红黑树的 5 大性质（插入 / 删除的“铁律”，必须背会）

1. 根节点是**黑色**；
2. 所有叶子节点（NIL 节点，空节点）是**黑色**；
3. 若一个节点是**红色**，则它的两个子节点必是**黑色**（无“红红冲突”）；
4. 从任意节点到其所有叶子节点的路径中，**黑色节点个数相同**（黑高相等）；
5. 新插入的节点默认是**红色**（减少破坏性质的概率）。

记忆口诀：“根黑、叶黑、红子黑、黑高同、插红”。

2. 红黑树的插入流程（核心：修复“红红冲突”）

插入步骤：“二叉搜索树插入→标记红色→判断是否破坏性质→修复”。

关键前提：

- 冲突场景：只有“父节点是红色”时，才会破坏性质 3（红红冲突）（父黑则无冲突，直接结束）；
- 修复核心：看“叔叔节点”（父节点的兄弟）的颜色，分 3 种情况。

3 种修复情况（设插入节点为 X，父节点为 P，祖父为 G，叔叔为 U）：

情况 1：叔叔 U 是红色（简单，调色即可）

- 操作：P→黑，U→黑，G→红；
- 原因：中和红色，保持黑高不变，消除红红冲突；
- 注意：若 G 是根节点，需再将 G→黑（恢复性质 1）。

情况 2：叔叔 U 是黑色，且 X 是右子节点（先旋转，再调色）

- 操作：先以 P 为轴，向左旋转（将 X 转为左子节点）→转化为情况 3；
- 原因：X 在右边，无法直接调色，旋转后变为左子节点，方便后续处理。

情况 3：叔叔 U 是黑色，且 X 是左子节点（调色 + 旋转）

- 操作：P→黑，G→红，以 G 为轴，向右旋转；
- 原因：调色消除冲突，旋转保持黑高不变。

记忆口诀：“插红找冲突，父红看叔叔；叔红全调色，叔黑看左右；右子先左转，左子调色右转”。

3. 红黑树的删除流程（考试侧重插入，删除简化记）

删除步骤：“二叉搜索树删除→替换节点→判断是否破坏性质→修复”。

- 核心问题：删除黑色节点会破坏性质 4（黑高不等），删除红色节点无影响；
- 修复核心：给“缺黑”的路径补黑，分“兄弟是红 / 黑”“兄弟的子节点颜色”等情况；

- 考试速记：“删黑补黑，看兄弟，调色 / 旋转”（无需记细节，重点在插入）。

1. B 树 / B + 树核心

- B 树： m 阶定范围，插入溢则分裂，中间上移，等高；IO = 树高，适合随机查；
- B + 树：叶子存全集，非叶是索引，分裂复制不上移，叶子链表，适合范围查；IO = 树高。

2. 红黑树核心

- 5 性质：“根黑、叶黑、红子黑、黑高同、插红”；
- 插入修复：“看叔叔，调色 / 旋转”。

3. 考试易错点

- B 树分裂：中间关键码上移，子节点不含该关键码；
- B + 树分裂：中间关键码复制，子节点仍含该关键码；
- 红黑树插入：新节点必红，冲突只在父红时出现；
- IO 次数：B 树 / B + 树的树高 = IO 次数，红黑树适合内存（IO 少），B 树 / B + 树适合外存（IO 少）。

1224习题课

1. 第八章作业重新看一下，1、3都错了qwq，具体过程
2. BST怎么求一个集合中多少元素 $>x$

往年题

21

1. 图遍历得到树：只要访问过的点就不再展开；
2. 子树：包括子节点的子节点！
3. 建最大/最小堆：比较次数最差 $O(n)$ ，最多大概 $2n$ 次（下移比2次
4. 要求装载因子之后这个可以直接计算得出总的散列长度，可以比mod的数大
5. 散列表查找失败，本质上意思就是说我们查的这个关键字不在散列表中，需要用对应的探查方法遇到空为止我们才知道失败；懂了，所以我们是默认每种mod K（hash所取的值）的余数概率相同，所以只有7种情况，每种查一遍，然后/7就行了。

而查找成功也很简单，就是说

6.

逐层拆解广义表 P 的结构，结合 head/tail 的规则一步步定位到 d。

第一步：先明确广义表 P 的结构

题目中 P 的定义是： $P=(L1:(a,b,L2), L2:(L1,b,c), L3:(c,d,L2))$ 即 P 是一个广义表，包含 3 个子表元素： L_1 、 L_2 、 L_3 （d 在 L_3 里）。

第二步：先定位到 L_3 （d 所在的子表）

要找到 d，首先得从 P 中取出 L_3 ，需要用 tail 去掉 P 的前两个元素，再用 head 取到 L_3 ：

1. 第一次 tail(P)：去掉 P 的第一个元素 L_1 ，得到子表 (L_2, L_3) ；
2. 第二次 tail(tail(P))：去掉 (L_2, L_3) 的第一个元素 L_2 ，得到子表 (L_3) ；
3. head(tail(tail(P)))：取 (L_3) 的第一个元素，得到 L_3 （此时拿到了 d 所在的子表）。

第三步：从 L_3 中取出 d

L_3 的结构是 $L_3=(c, d, L_2)$ ，d 是 L_3 的第二个元素，需要用 tail 去掉 L_3 的第一个元素，再用 head 取到 d：

1. tail(L_3)：去掉 L_3 的第一个元素 c，得到子表 (d, L_2) ；
2. head(tail(L_3))：取 (d, L_2) 的第一个元素，得到 d。